

JavaScript Problem Set

50 Problems · 10 Days · 5 Problems per Day

Difficulty: **Easy** **Medium** **Hard**

Solve 5 problems daily — practice, test, improve.

Day 1 — Variables, Data Types & Operators

Problem 1: Swap Two Variables [Easy]

Description: Write a function that swaps the values of two variables without using a third variable.

Example:

Input: `a = 5, b = 10` Output: `a = 10, b = 5`

Hint: Try using destructuring or arithmetic operators.

Problem 2: Check Even or Odd [Easy]

Description: Write a function `isEven(n)` that returns `true` if a number is even, and `false` if it is odd.

Example:

Input: `4` → Output: `true` Input: `7` → Output: `false`

Hint: Use the modulus (%) operator.

Problem 3: Find the Largest of Three Numbers [Easy]

Description: Write a function `largest(a, b, c)` that returns the largest of three numbers.

Example:

Input: `3, 7, 5` → Output: `7`

Hint: Use `Math.max()` or `if-else` conditions.

Problem 4: Celsius to Fahrenheit [Easy]

Description: Write a function `toFahrenheit(celsius)` that converts a Celsius temperature to Fahrenheit.

Example:

Input: 0 → Output: 32 Input: 100 → Output: 212

Hint: Formula: $(C \times 9/5) + 32$

Problem 5: Check Positive, Negative or Zero [Easy]

Description: Write a function `checkSign(n)` that returns 'positive', 'negative', or 'zero' based on the value of `n`.

Example:

Input: -5 → Output: 'negative' Input: 0 → Output: 'zero'

Hint: Use if-else if-else statements.

Day 2 — Strings & String Methods

Problem 6: Reverse a String [Easy]

Description: Write a function `reverseString(str)` that returns the reverse of a given string.

Example:

Input: 'hello' → Output: 'olleh' Input: 'world' → Output: 'dlrow'

Hint: Use `split()`, `reverse()`, and `join()`.

Problem 7: Count Vowels in a String [Easy]

Description: Write a function `countVowels(str)` that counts and returns the number of vowels (a, e, i, o, u) in a string.

Example:

Input: 'hello' → Output: 2 Input: 'javascript' → Output: 3

Hint: Use a loop or `match()` with a regular expression.

Problem 8: Check Palindrome [Easy]

Description: Write a function `isPalindrome(str)` that returns true if the string reads the same forwards and backwards.

Example:

Input: 'racecar' → Output: true Input: 'hello' → Output: false

Hint: Compare the string to its reverse.

Problem 9: Capitalize First Letter of Each Word [Easy]

Description: Write a function `titleCase(str)` that capitalizes the first letter of every word in a string.

Example:

Input: 'hello world' → Output: 'Hello World'

Hint: Use `split(' '), map()`, and `join(' ')`.

Problem 10: Count Occurrences of a Character [Easy]

Description: Write a function `countChar(str, char)` that returns how many times a character appears in a string.

Example:

Input: 'banana', 'a' → Output: 3

Hint: Use `split(char).length - 1` or a loop.

Day 3 — Arrays & Array Methods

Problem 11: Find the Sum of an Array [Easy]

Description: Write a function `sumArray(arr)` that returns the sum of all numbers in an array.

Example:

Input: `[1, 2, 3, 4, 5]` → Output: 15

Hint: Use `reduce()` or a `for` loop.

Problem 12: Find Maximum Value in Array [Easy]

Description: Write a function `findMax(arr)` that returns the largest number in an array without using `Math.max()`.

Example:

Input: `[3, 1, 7, 2, 9]` → Output: 9

Hint: Loop through and track the largest value found.

Problem 13: Remove Duplicates from Array [Easy]

Description: Write a function `removeDuplicates(arr)` that returns a new array with duplicate values removed.

Example:

Input: `[1, 2, 2, 3, 3, 4]` → Output: `[1, 2, 3, 4]`

Hint: Use `Set` or `filter()` with `indexOf()`.

Problem 14: Flatten a Nested Array [Medium]

Description: Write a function `flattenArray(arr)` that flattens one level of a nested array.

Example:

Input: `[1, [2, 3], [4, 5]]` → Output: `[1, 2, 3, 4, 5]`

Hint: Use flat() or reduce() with concat().

Problem 15: Chunk an Array [Medium]

Description: Write a function chunkArray(arr, size) that splits an array into chunks of a given size.

Example:

Input: [1,2,3,4,5], 2 → Output: [[1,2],[3,4],[5]]

Hint: Use a while loop with slice().

Day 4 — Objects & Loops

Problem 16: Count Object Properties [Easy]

Description: Write a function countProperties(obj) that returns the number of properties in an object.

Example:

Input: {a: 1, b: 2, c: 3} → Output: 3

Hint: Use Object.keys().length.

Problem 17: Merge Two Objects [Easy]

Description: Write a function mergeObjects(obj1, obj2) that merges two objects into one. If keys conflict, the second object's values win.

Example:

Input: {a:1}, {b:2} → Output: {a:1, b:2}

Hint: Use the spread operator or Object.assign().

Problem 18: FizzBuzz [Easy]

Description: Write a function `fizzBuzz(n)` that prints numbers from 1 to `n`. For multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', multiples of both print 'FizzBuzz'.

Example:

Input: 15 Output: 1,2,Fizz,4,Buzz,Fizz,7,8,Fizz,Buzz,11,Fizz,13,14,FizzBuzz

Hint: Check divisibility with the % operator in the right order.

Problem 19: Invert an Object [Easy]

Description: Write a function `invertObject(obj)` that swaps the keys and values of an object.

Example:

Input: {a: 1, b: 2} → Output: {1: 'a', 2: 'b'}

Hint: Use `Object.entries()` and `reduce()`.

Problem 20: Find Duplicate Values in Array of Objects [Medium]

Description: Given an array of objects, write a function `findDuplicateNames(arr)` that returns names that appear more than once.

Example:

Input: [{name: 'Ali'}, {name: 'Sara'}, {name: 'Ali'}] Output: ['Ali']

Hint: Use a frequency map (object) to count occurrences.

Day 5 — Functions & Scope

Problem 21: Factorial (Recursive) [Easy]

Description: Write a recursive function `factorial(n)` that returns the factorial of a non-negative integer `n`.

Example:

Input: 5 → Output: 120 (5×4×3×2×1) Input: 0 → Output: 1

*Hint: Base case: $factorial(0) = 1$. Recursive case: $n * factorial(n-1)$.*

Problem 22: Fibonacci Sequence [Easy]

Description: Write a function `fibonacci(n)` that returns the `n`th number in the Fibonacci sequence.

Example:

Input: 6 → Output: 8 (0,1,1,2,3,5,8...)

Hint: Try both iterative and recursive approaches.

Problem 23: Create a Counter with Closure [Medium]

Description: Write a function `makeCounter()` that returns an object with `increment`, `decrement`, and `getCount` methods using closure.

Example:

```
const c = makeCounter(); c.increment(); c.increment(); c.getCount(); // 2
```

Hint: Use a variable inside the outer function that inner functions can access.

Problem 24: Curry a Function [Medium]

Description: Write a function `curry(fn)` that converts a function of two arguments into a curried version.

Example:

```
const add = curry((a,b) => a+b); add(2)(3); // 5
```

Hint: Return a function from inside a function.

Problem 25: Memoize a Function [Medium]

Description: Write a function `memoize(fn)` that caches the results of a function so repeated calls with the same input return the cached result.

Example:

```
const memoAdd = memoize(n => n + 10);memoAdd(5); // 15 (computed)memoAdd(5);  
// 15 (from cache)
```

Hint: Use an object as a cache inside the outer function.

Day 6 — ES6+ Features

Problem 26: Destructure and Rename [Easy]

Description: Given a user object, use destructuring to extract `firstName`, `lastName`, and `age`, renaming `firstName` to `name`.

Example:

```
const user = {firstName:'Sara', lastName:'Khan', age:25};// Extract:  
name='Sara', lastName='Khan', age=25
```

Hint: Use: `const { firstName: name, lastName, age } = user;`

Problem 27: Merge Arrays with Spread [Easy]

Description: Write a function `mergeArrays(...arrays)` that accepts any number of arrays and merges them into one using the spread operator.

Example:

```
Input: [1,2], [3,4], [5]Output: [1,2,3,4,5]
```

Hint: Use rest parameters and spread inside `reduce` or `flat`.

Problem 28: Promise Chain [Medium]

Description: Write a function `delay(ms)` that returns a Promise that resolves after `ms` milliseconds. Then chain two delays: first 1 second, then 2 seconds, logging a message after each.

Example:

```
delay(1000).then(() => { console.log('1 sec'); return delay(2000);
}).then(() => console.log('3 sec total'));
```

Hint: Use new Promise with setTimeout inside.

Problem 29: Async/Await Fetch Simulation [Medium]

Description: Write an async function `getUserData(id)` that simulates fetching user data by awaiting a Promise that resolves after 500ms with a mock user object.

Example:

```
await getUserData(1); // returns {id:1, name:'Test User'}
```

Hint: Use async/await with a Promise that wraps setTimeout.

Problem 30: Optional Chaining & Nullish Coalescing [Easy]

Description: Given a nested object that may have missing properties, safely access a deeply nested value using optional chaining (`?.`) and provide a default using nullish coalescing (`??`).

Example:

```
const user = {profile: null}; const city = user?.profile?.address?.city ??
'Unknown'; // Output: 'Unknown'
```

Hint: Chain `?.` for each level; use `??` for the fallback.

Day 7 — DOM & Events (Conceptual/Logic)

Problem 31: Debounce Function [Medium]

Description: Write a `debounce(fn, delay)` function that delays invoking `fn` until after `delay` milliseconds have elapsed since the last time it was called.

Example:

```
const debouncedSearch = debounce(search, 300); // Fires only once after user stops typing for 300ms
```

Hint: Use `clearTimeout` and `setTimeout`.

Problem 32: Throttle Function [Medium]

Description: Write a `throttle(fn, limit)` function that ensures `fn` is called at most once every `limit` milliseconds.

Example:

```
const throttledScroll = throttle(onScroll, 200); // Fires at most once every 200ms during scroll
```

Hint: Track the last call time with `Date.now()`.

Problem 33: Deep Clone an Object [Medium]

Description: Write a function `deepClone(obj)` that returns a deep copy of a plain object without using `JSON.parse/JSON.stringify`.

Example:

```
const a = {x: {y: 1}}; const b = deepClone(a); b.x.y = 99; // a.x.y is still 1
```

Hint: Use recursion and check for object/array types.

Problem 34: Event Emitter [Medium]

Description: Build a simple EventEmitter class with on(event, listener), emit(event, ...args), and off(event, listener) methods.

Example:

```
const emitter = new EventEmitter(); emitter.on('greet', name =>
  console.log('Hello ' + name)); emitter.emit('greet', 'Sara'); // Hello Sara
```

Hint: Store listeners in an object where keys are event names and values are arrays of functions.

Problem 35: Implement Array.prototype.map from Scratch [Medium]

Description: Write a function myMap(arr, callback) that replicates the behavior of Array.prototype.map without using the built-in map().

Example:

```
myMap([1,2,3], x => x * 2); // Output: [2, 4, 6]
```

Hint: Loop through the array, apply the callback to each element, push results to a new array.

Day 8 — Error Handling & Advanced Patterns

Problem 36: Safe JSON Parse [Easy]

Description: Write a function safeJsonParse(str) that tries to parse a JSON string and returns the parsed value, or null if parsing fails — without throwing an error.

Example:

```
safeJsonParse('{"a":1}'); // {a: 1} safeJsonParse('bad json'); // null
```

Hint: Use try/catch inside the function.

Problem 37: Retry a Promise [Medium]

Description: Write a function `retry(fn, times)` that calls an async function `fn` up to `times` times, retrying if it throws an error. Resolves on first success, rejects after all attempts fail.

Example:

```
await retry(unstableFetch, 3); // Tries up to 3 times before failing
```

Hint: Use a loop with try/catch; only throw after all retries are exhausted.

Problem 38: Implement Promise.all from Scratch [Hard]

Description: Write a function `myPromiseAll(promises)` that behaves like `Promise.all` — resolves with an array of results when all resolve, rejects immediately if any rejects.

Example:

```
myPromiseAll([p1, p2, p3]).then(results => console.log(results));
```

Hint: Track resolved count and results array; reject on first failure.

Problem 39: Flatten Object (Deep) [Medium]

Description: Write a function `flattenObject(obj)` that takes a deeply nested object and returns a flat object with dot-notation keys.

Example:

```
Input: {a: {b: {c: 1}}} Output: {'a.b.c': 1}
```

Hint: Use recursion; build the key by joining parent keys with dots.

Problem 40: Group Array by Property [Medium]

Description: Write a function `groupBy(arr, key)` that groups an array of objects by a given property key.

Example:

```
groupBy([{'type': 'fruit', name: 'apple'}, {'type': 'veg', name: 'carrot'}, {'type': 'fruit', name: 'mango'}], 'type') // {fruit: [...], veg: [...]}
```

Hint: Use reduce() and build an object where each key maps to an array.

Day 9 — Algorithms & Problem Solving I

Problem 41: Two Sum [Easy]

Description: Given an array of numbers and a target, write a function `twoSum(arr, target)` that returns the indices of the two numbers that add up to the target.

Example:

Input: `[2, 7, 11, 15]`, target = 9 Output: `[0, 1]` (because $2 + 7 = 9$)

Hint: Use a hash map to store visited values and their indices.

Problem 42: Check Anagram [Easy]

Description: Write a function `isAnagram(str1, str2)` that returns true if the two strings are anagrams of each other.

Example:

Input: `'listen', 'silent'` → Output: true Input: `'hello', 'world'` → Output: false

Hint: Sort both strings and compare, or use a character frequency map.

Problem 43: Find Missing Number [Easy]

Description: Given an array of $n-1$ integers from 1 to n with one number missing, write a function `findMissing(arr, n)` to find the missing number.

Example:

Input: `[1, 2, 4, 5]`, $n=5$ → Output: 3

Hint: Use the formula: $\text{expected sum} = n(n+1)/2$, then subtract actual sum.*

Problem 44: Valid Parentheses [Medium]

Description: Write a function `isValidParentheses(str)` that returns true if the string has valid, balanced parentheses, brackets, and braces.

Example:

Input: `'()[]{}'` → Output: `true` Input: `'([)]'` → Output: `false`

Hint: Use a stack (array). Push opening brackets, pop and compare for closing ones.

Problem 45: Binary Search [Medium]

Description: Write a function `binarySearch(arr, target)` that searches a sorted array and returns the index of the target, or -1 if not found.

Example:

Input: `[1,3,5,7,9]`, `target=7` → Output: `3`

Hint: Use left and right pointers; check the middle element each iteration.

Day 10 — Algorithms & Problem Solving II

Problem 46: Count Words in a Sentence [Easy]

Description: Write a function `wordCount(sentence)` that returns an object with each word as a key and its frequency as the value.

Example:

Input: `'the cat sat on the mat'` Output: `{the: 2, cat: 1, sat: 1, on: 1, mat: 1}`

Hint: Split by spaces, then reduce into a frequency object.

Problem 47: Longest Word in a Sentence [Easy]

Description: Write a function `longestWord(sentence)` that returns the longest word in a sentence. If there's a tie, return the first one.

Example:

Input: 'The quick brown fox' Output: 'quick'

Hint: Split the sentence and use `reduce()` to track the longest.

Problem 48: Rotate an Array [Medium]

Description: Write a function `rotateArray(arr, k)` that rotates an array to the right by `k` steps.

Example:

Input: `[1,2,3,4,5]`, `k=2` Output: `[4,5,1,2,3]`

Hint: Use `slice` and `concat`, or reverse the whole array then parts.

Problem 49: Roman to Integer [Medium]

Description: Write a function `romanToInt(s)` that converts a Roman numeral string to an integer.

Example:

Input: 'III' → Output: 3 Input: 'IX' → Output: 9 Input: 'LVIII' → Output: 58

Hint: Map each symbol to its value; if a smaller value comes before a larger one, subtract it.

Problem 50: Pascal's Triangle Row [Medium]

Description: Write a function `pascalRow(n)` that returns the `n`th row of Pascal's Triangle as an array.

Example:

Input: 0 → Output: [1] Input: 3 → Output: [1,3,3,1] Input: 4 → Output:
[1,4,6,4,1]

Hint: Each element is the sum of the two elements above it from the previous row.
